SOFTWARE ENGINEERING · LECTURE 3

# Scan to follow along

Open today's slides on your own phone or laptop — then you can run real code yourself.

`hassanmiir.github.io/cpp-course/lecture3`

**Dr. Mir Hassan**

Associate Professor in Artificial Intelligence and Trustworthy Digital Systems



📱 Point your camera here

👁 0 opened today

LECTURE 3 · OPERATORS & EXPRESSIONS

# Now let's do maths with our data

Last time you *stored* a student's marks. Today you'll *compute* with them — averages, percentages, and the pass/fail decision itself.

A 2-hour hands-on session · you'll build a working calculator

THE BIG IDEA

# An expression is a question the computer answers

You write a small sum; C++ works it out and hands back **one value**.



```
int total = 80 + 60;      // expression on the right → 140
double avg = total / 2.0;   // another expression → 70.0
```

`80 + 60` → [ 140 ]

An **operator** ( `+` , `/` ...) combines **operands** (the values) into a result.

PART 1

# The arithmetic operators

Five symbols for the everyday maths your programs need.

- Add, subtract, multiply, divide
- The remainder operator — the surprising one
- Run a calculator — live



REFERENCE

# Four everyday operators

| + | - | * | / |
|---|---|---|---|
| Add | Subtract | Multiply | Divide |
| 7 + 2 → 9 | 7 - 2 → 5 | 7 * 2 → 14 | 7 / 2 → ? |

**Hold on**

What does `7 / 2` give? Not 3.5, if both numbers are `int`. That trap is the next slide.

THE DIVISION TRAP

# Two whole numbers divide to a whole number

```
7 / 2         // → 3    (not 3.5! the .5 is dropped)
7.0 / 2       // → 3.5  (one decimal makes it a real divide)
7 / 2.0       // → 3.5
(double)7/2   // → 3.5  (force it)
```

### int ÷ int = int

C++ throws away the fraction. 7/2 is 3, 1/2 is 0.

### The fix

Make **one side** a decimal: `/ 2.0`. Now you keep the decimals.

**Why it matters**

An average of marks with `/ 2` silently loses accuracy. Grades must use `/ 2.0`.

NEW OPERATOR

# Modulo `%` — the leftover

`%` gives what's **left over** after dividing whole numbers.

```
17 / 5   // → 3   (how many fit)
17 % 5   // → 2   (what's left over)
```

### Even or odd?

`n % 2 == 0` → even. The classic use.

### Split into groups

30 students, groups of 4 → `30 % 4 = 2` left over.

WORK IT OUT · ON PAPER · 2 MIN

# Fill in the table — no computer

Both numbers are `int`. Write the quotient `/` and the remainder `%` for each row.

| | | |
|---|---|---|
| `13 / 4` | quotient = ? | 13 % 4 remainder = ? |
| `20 / 5` | quotient = ? | 20 % 5 remainder = ? |
| `7 / 10` | quotient = ? | 7 % 10 remainder = ? |

✏️ **Pens down when you have all six. Then we check — and run it.**

THE ANSWER

# Check your six

| | | | |
|---|---|---|---|
| `13 / 4` | = **3** | 13 % 4 = **1** | (4×3 + 1 = 13) |
| `20 / 5` | = **4** | 20 % 5 = **0** | (divides evenly) |
| `7 / 10` | = **0** | 7 % 10 = **7** | (10 doesn't fit — all 7 left over) |

**The surprising one**

`7 / 10 = 0` catches everyone. Fewer than one "10" fits, so the whole-number quotient is 0 and the remainder is the lot.

YOUR TURN · LIVE CODE RUN IT!

# A two-number calculator

Run it. Then change the numbers — try 7 and 2, and watch `/` vs `%`. (Input pre-filled: 17 5)

main.cpp ▶ Run ↺ Open in OnlineGDB ▸

```
#include <iostream>
using namespace std;

int main() {
    int a, b;
    cin >> a >> b;
    cout << "Sum:        " << a + b << endl;
    cout << "Difference: " << a - b << endl;
    cout << "Product:    " << a * b << endl;
    cout << "Quotient:   " << a / b << endl;
    cout << "Remainder:  " << a % b << endl;
    return 0;
```

OUTPUT Ready

Click ▶ Run to see the output here.

QUICK CHECK THINK!

# What does `9 / 4` print, if both are `int`?

A 2.25

B 2

C 3

D 1

PART 2

# Comparing & combining

Operators that answer true or false — the seed of every decision.

- Compare two values
- Combine conditions with and / or
- The `=` vs `==` mix-up



REFERENCE

# Comparisons give a `bool`

Each of these asks a yes/no question and returns `true` or `false`.

| `>` `<` | `>=` `<=` | `==` ? | `!=` |
|---|---|---|---|
| greater / less | or equal to | is equal? | is not equal? |

**The #1 beginner bug**

`=` puts a value in a box. `==` asks a question. `mark == 50` checks; `mark = 50` overwrites!

COMBINE CONDITIONS

# and `&&`, or `||`, not `!`

**`&&` — both must be true**

`mark >= 50 && attended` → passed *and* showed up.

**`||` — at least one is true**

`isStaff || isAdmin` → either one grants access.

**`!` — flips it**

`!isOpen` → true when it's closed.



These build the pass/fail rule you'll use in Lecture 4.

WORK IT OUT ON PAPER · 2 MIN

# True or false? Work each one out

Take `mark = 72`, `attended = true`, `isStaff = false`. Write T or F for each.

```
mark >= 50                  ?
mark >= 50 && attended      ?
mark > 90 || isStaff        ?
!attended                   ?
```

✏️ **Reason it step by step — evaluate each side, then combine.**

THE ANSWER

# How each one resolves

| | | |
|---|---|---|
| `mark >= 50` | `true` | 72 ≥ 50 ✓ |
| `mark >= 50 && attended` | `true` | true AND true |
| `mark > 90 \|\| isStaff` | `false` | false OR false — needs *one* true |
| `!attended` | `false` | attended is true, ! flips it |

**That's the pass rule**

Row 2 is exactly the condition your program builds next — a `bool` straight from the operators you just evaluated.

YOUR TURN · LIVE CODE RUN IT!

# Did the student pass?

A real condition: average ≥ 50 **and** attended. Change the marks and the attendance flag. (Input: 80 55 1)

main.cpp ▶ Run ↺ Open in OnlineGDB ▸

```
#include <iostream>
using namespace std;

int main() {
    int mark1, mark2;
    bool attended;
    cin >> mark1 >> mark2 >> attended;

    double average = (mark1 + mark2) / 2.0;   // keep the decimals!
    bool passed = (average >= 50) && attended;

    cout << "Average: " << average << endl;
```

OUTPUT Ready

Click ▶ Run to see the output here.

QUICK CHECK THINK!

# Which line *checks* whether `score` is 100?

**A** `score = 100`

**B** `score == 100`

**C** `score =< 100`

**D** `score ! 100`

PART 3

# Order of operations

Which part runs first — and how brackets put you in control.



× and ÷ before + and −

Brackets always win

THE RULE

# C++ follows the maths order you know

```
2 + 3 * 4         // → 14, not 20  (× first)
(2 + 3) * 4       // → 20          (brackets first)
80 + 60 / 2       // → 110, not 70 (÷ first!)
(80 + 60) / 2     // → 70          (what you meant)
```

`%` ranks with `*` and `/`; comparisons and `&&` / `||` come after the arithmetic.

### When in doubt, bracket

Extra `( )` never hurt — they make your intent obvious and bug-proof.

### The average bug

`a + b / 2` only halves `b`. You need `(a + b) / 2.0`.

WORK IT OUT ON PAPER · 2 MIN

# This average is wrong. Fix it.

A student scored `80` and `60`. This line should give `70.0` — but it doesn't.

```
double avg = 80 + 60 / 2;
```

1 · What does this actually compute? ?

2 · Rewrite it so it gives 70.0

✏️ **Two things are wrong — the order *and* the decimals. Find both.**

THE ANSWER

# Two bugs, one line

```
// WRONG — / runs before +, so:
80 + 60 / 2   // = 80 + 30 = 110  ✗

// RIGHT — bracket the sum, keep decimals:
(80 + 60) / 2.0   // = 140 / 2.0 = 70.0  ✓
```

### Bug 1 — order

`/` beats `+`, so only 60 got halved. Brackets fix it.

### Bug 2 — decimals

`/ 2.0` keeps the fraction, so averages never silently round.

QUICK CHECK THINK!

# What is `10 + 20 * 2`?

A 60

B 50

C 40

PART 4

# Build it: a grade calculator

Put every operator together — the real piece Group B's gradebook needs.

- Weighted final mark
- A pass/fail bool from your conditions
- Work with a partner



PAIR CHALLENGE · USE YOUR BRAIN 15 MIN

# The weighted grade

A course grade is **40% coursework + 60% exam**. Compute it and decide pass/fail:

1. Read **coursework** and **exam** marks (0–100).
2. Weight them: `final = cw*0.4 + exam*0.6` — watch the decimals!
3. Decide: a `bool passed` — is `final >= 50`?
4. Print the final mark and whether they passed.

15:00

❚❚ Pause ↺ +1 min

**The catch**

Use `0.4` and `0.6` (decimals) so the weighting isn't rounded to zero — the division trap, again in disguise.

WORK IT OUT FIRST ON PAPER · 3 MIN

# Predict the result before you write a line

Coursework = `70`, exam = `40`, rule = `40% + 60%`, pass = `>= 50`. Work it out by hand.

1 · coursework × 0.4 = ?

2 · exam × 0.6 = ?

3 · final mark (add them) = ?

4 · passed? (final ≥ 50) = ?

✏️ **Agree your four answers in your pair. Then code it — and check the machine matches your paper.**

YOUR TARGET

# What your program must print

coursework × 0.4 = **28.0** 70 × 0.4

exam × 0.6 = **24.0** 40 × 0.6

final = **52.0** 28 + 24

passed = **true** 52 ≥ 50 ✓

**Now build it**

You know the answer. Write the code on the next slide and make the computer agree with your paper.

PAIR CHALLENGE · BUILD HERE RUN IT!

# Your grade calculator

Starter reads the marks. **You add** the weighted final and the pass check. (Input pre-filled: 70 40)

main.cpp ▶ Run ↺ Open in OnlineGDB ▸

```
#include <iostream>
using namespace std;

int main() {
    int coursework, exam;
    cin >> coursework >> exam;

    // TODO 1: double final = coursework * 0.4 + exam * 0.6;
    // TODO 2: bool passed = final >= 50;

    cout << "==== GRADE ====" << endl;
    cout << "Coursework: " << coursework << endl;
```

OUTPUT Ready

```
Click ▶ Run to see the output here.
```

ONE SOLUTION COMPARE YOURS

# How the calculator works

```cpp
double final = coursework * 0.4 + exam * 0.6;  // decimals keep accuracy
bool passed = final >= 50;

cout << "==== GRADE ====" << endl;
cout << "Coursework: " << coursework << endl;
cout << "Exam:       " << exam << endl;
cout << "Final:      " << final << endl;
cout << "Passed:     " << passed << endl;
cout << "===============" << endl;
```

### Weighting = multiply + add

70·0.4 + 40·0.6 = 28 + 24 = **52.0**.

### `final >= 50` → true

A comparison gives the pass bool — no "if" needed yet.

### That's a gradebook row

Group B computes exactly this for every student.

RECAP

# What you can do now

### Arithmetic

The five operators, the `int ÷ int` trap, and `%` for remainders.

### Logic

Comparisons and `&&` / `||` that produce a `bool` — and precedence you control with brackets.

**Milestone**

Your programs now turn data into answers — a working calculator and a real pass/fail rule.

KEEP THESE WORDS

# Today's operators — click any one for a reminder

| | | | |
|---|---|---|---|
| `%` ? Remainder | `==` ? Is equal? | `&&` ? And | `\|\|` ? Or |
| `!` ? Not | `/ 2.0` ? Real divide | `( )` ? Order | `expression` ? Makes a value |

BEFORE NEXT TIME

# Practice (15–20 min)

### Temperature converter

Read Celsius, print Fahrenheit: `F = C * 9.0 / 5.0 + 32`. Mind the decimals.

### Odd or even

Read a number; print whether `n % 2 == 0`. (It prints 1 or 0 for now.)

### Extend your calculator

Add the weighted grade to your project folder — Group B will reuse it.

NEXT LECTURE

# Lecture 4 — Making Decisions

Today your pass check printed `1` or `0`. Next time, `if` turns that bool into real branches — **"PASS"** or **"FAIL"**, different paths for different data.



- `if` / `else` — choose what runs
- Turn today's conditions into decisions
- A grade classifier: A, B, C, fail



# You built a calculator.

Operators turn stored data into real answers — grades, averages, pass or fail. Next: decisions.

Dr. Mir Hassan · Associate Professor in AI & Trustworthy Digital Systems